MINI-REACT ENGINEERING LAB · SOFTWARE DESIGN DESCRIPTION

기준: SRS v1.0 (2026-11-07) · IEEE 1016-2009

미니 React 소프트웨어 설계 다이어그램 리포트 (SDD)

Virtual DOM 부분 업데이트, 5대 패치 유형(Diff/Patch), History 세이브 슬롯, 리액티브 훅(Hooks) 엔진 및 7계층 검증 하네스 아키텍처

문서 상태	작성 일자	표준 정합도	구현 언어
Draft v0.3 (ADR 리뷰)	2026-11-09	IEEE 1016 (26/30점, 87%)	TypeScript (ADR-0001)

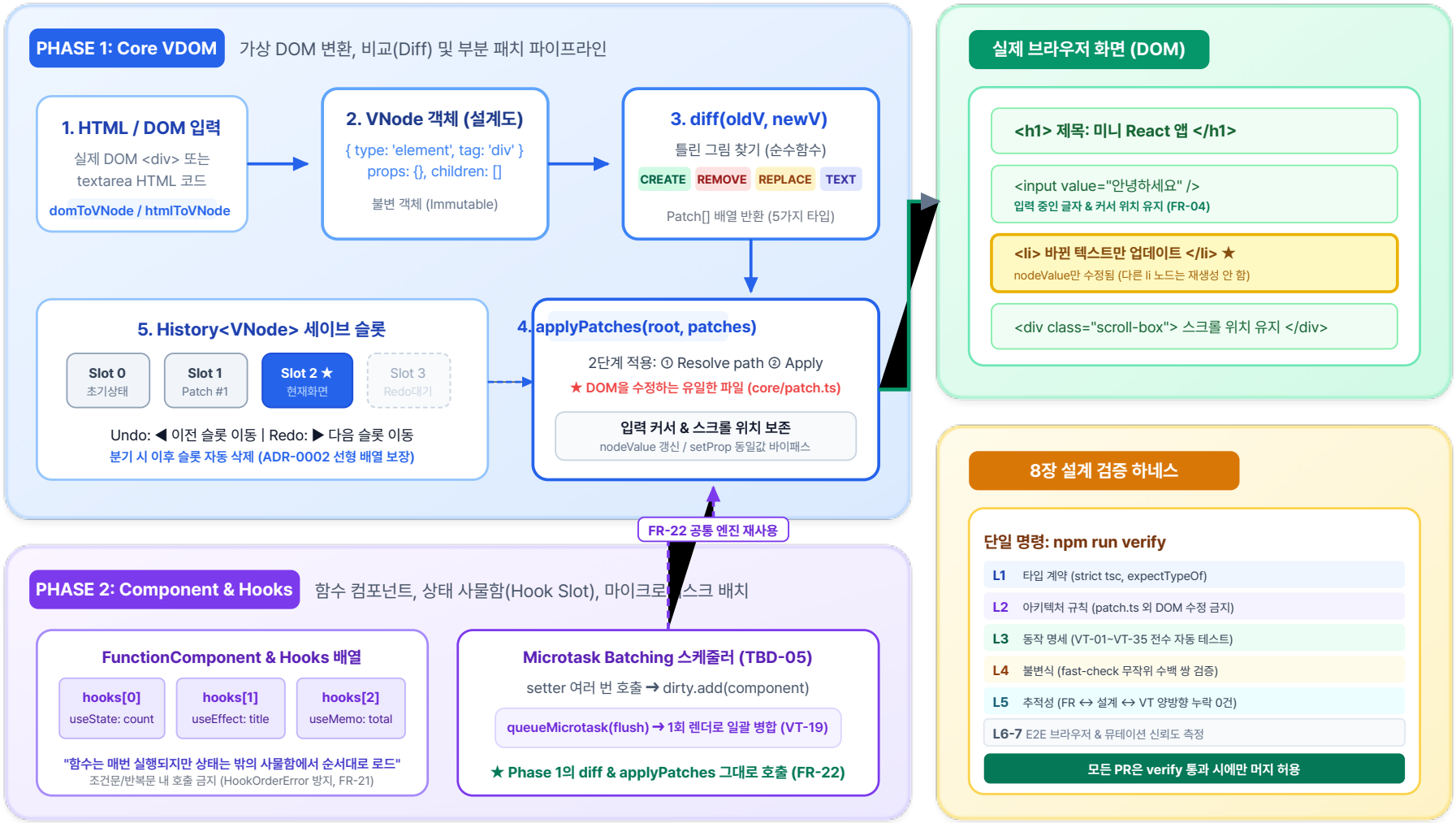
01 0장 한 장 설계 지도 (Architecture Blueprint)

0장 한 장 설계 지도

미니 React 핵심 아키텍처 맵 (Architecture Blueprint)

IEEE 1016 / ISO 29148 정합 설계

전체를 다시 그리지 않고 바뀐 블록만 골라 교체하는 Virtual DOM & 리액티브 훅 엔진



부분 업데이트 (FR-04)
innerHTML 전체 교체 대신 `core/patch.ts`에서 바뀐 노드만 선별 수정하여 입력값과 스크롤을 완벽히 보존합니다.

사물함 훅 & 엔진 재사용 (FR-22)
함수는 새로 실행되지만 상태는 외부 `hooks[]`에 순서대로 유지되며, 변경 시 Phase 1 diff/patch를 그대로 호출합니다.

7계층 하네스 보증 (8장)
팀원 및 AI가 작성한 모든 코드는 `npm run verify`를 통해 아키텍처 규칙과 VT-01~35를 100% 자동 통과해야 합니다.

02 아키텍처 결정 레코드 (ADR-0001 ~ ADR-0009) 전수 명세

아키텍처 결정 레코드 (ADR 1~9 전수 완비)

우리가 고민하고 결정한 9가지 아키텍처 원칙 (ADR-0001 ~ 0009)

"왜 이렇게 만들었나요?"라는 질문에 가장 확실하고 우아하게 답변할 수 있는 기술적 근거와 트레이드오프 기록입니다. 중·고등학생도 한 번에 이해할 수 있는 직관적인 비유와 실제 PR #47 변경점 링크를 함께 제공합니다.

참조 GitHub 풀리퀘스트
PR #47 (changes)
9개 ADR 전문 수록 완료

상태 필터: 전체 9건 | 확정 (Accepted) | 제안/승인대기 (Proposed)

ADR 제목, 비유, 내용 검색...

ADR-0001 Accepted (확정) § 1.0, 2.1
프로그래밍 언어 선택 (TypeScript strict)
실행 전 오타와 실수를 100% 잡아내기 위해 TypeScript 채택
🔪 치수가 1mm라도 틀리면 "삐-!" 경고음을 내며 미리 ... #10

ADR-0002 Proposed (팀 승인 대기) § 3.5, 3.6
실행 취소(Undo) 후 새로 작업할 때 미래 기록 처리 (TBD-01)
과거로 돌아가 새 작업을 하면 원래 미래는 지우고 깔끔한 한 줄로 유지

ADR-0002 Proposed (팀 승인 대기) SDD § 3.5, 3.6

실행 취소(Undo) 후 새로 작업할 때 미래 기록 처리 (TBD-01)

관련 깃허브 이슈
Issue #23, #25

🎬 영화처럼 과거로 시간 여행을 가서 역사를 바꾸면, 원래... #23, #25

ADR-0003 Proposed § 3.2, 3.6

테스트 화면 입력 방식 (TBD-02)

브라우저 오류를 막기 위해 코드 입력창 + 실시간 미리보기 분리

🪞 진짜 벽에 낙서하지 않고, 메모장에 글을 쓴 뒤 옆에 있... #27

ADR-0004 Proposed § 3.3

Diff 비교 기준: 번호 순서대로 비교하기 (TBD-04)

출석 번호 순서(위치 인덱스)대로 비교하는 단순하고 튼튼한 방식 채택

🧍 출석 번호(1번, 2번...) 순서대로 줄을 서서, 지난번 내 ... #11

ADR-0005 Proposed § 3.4

화면 고칠 때 2단계 분리 (스티커 붙이고 → 한 번에 교체)

순서 밀림 버그를 막기 위해 "스티커 먼저 붙이고 확인 후 한 번에 교체"

🏷️ 블록을 바로 빼지 않고, 뺄 블록들에 포스트잇 스티커를... #10

✨ **직관적 규칙 비유 (중·고등학생 눈높이)**

"🎬 영화처럼 과거로 시간 여행을 가서 역사를 바꾸면, 원래 있던 옛날 미래는 사라지고 새로운 미래가 열리는 규칙"

1. 부딪힌 문제와 고민 (왜 고민했는가?)

게임에서 3단계까지 갔다가 2단계로 되돌아왔는데, 여기서 새로운 길로 가면 원래 가봤던 3단계 미래는 어떻게 해야 할까요?

2. 우리의 해결책과 결정 (어떻게 풀었는가?)

원래 있던 미래는 깨끗이 지우고(Prune), 내가 새로 걷기 시작한 길만 새 역사로 기록합니다 (웹 브라우저의 "뒤로가기" 버튼 원리와 동일).

3. 결과와 좋은 점 (무엇을 얻었는가?)

미래 기록이 여러 갈래로 복잡하게 얽히지 않고 깔끔한 한 줄로 유지되어, "다시 실행(Redo)"을 눌렀을 때 헷갈리지 않습니다.

✨ **발표/Q&A 질문 공격 방어 멘트**

질문: 'Undo 후 새로 Patch할 때 미래 기록을 왜 트리로 보존하지 않고 지우나요?' → 답변: '사용자에게 브라우저 뒤로가기 버튼과 동일한 직관성을 주고, Redo의 의미를 단일 선형으로 유지하여 혼란을 방지하기 위함입니다.'

03 2장 시스템 계층 구조 및 폴더/파일 책임 명세도

모듈 경계 & 파일 책임 명세도

우리가 설계한 미니 React 폴더 & 파일 구조

"어떤 파일이 무슨 일을 하고, 왜 분리했는가?" — 코어 엔진(core), 리액티브 훅(component), 테스트 화면(pages), 자동화 하네스(tests), 설계 기록(docs/adr)의 5단계 질서를 한눈에 파악하세요.

관련 GitHub PR
PR #47 (changes)
Issues #10, #11, #12, #13, #23, #25, #27

1

단방향 의존성 (역참조 금지)

pages → component → core 순서로만 참조합니다. 하위 엔진(core)은 상위 계층(component, pages)의 코드를 절대로 import할 수 없습니다 (스마트폰 부품 공장 원리).

`NFR-03 검증 통과`

2

단 1명의 목수 (patch.ts)

시스템 전체에서 진짜 화면(DOM)을 직접 건드릴 수 있는 곳은 오직 **src/core/patch.ts** 단 1곳뿐입니다.

`L2 ESLint 규칙 강제`

3

루트 컴포넌트 단일 상태 소유

상태(State)는 최상위 루트 컴포넌트만 소유하고, 하위 자식 컴포넌트는 Props만 받는 순수 함수입니다 (자식에서 훅 호출 시 에러 차단).

`ADR-0007 확정`

4

세이브 슬롯 절단 (타임머신)

과거로 Undo한 후 새로 Patch가 일어나면, 과거 이후에 있던 원래 미래 슬롯들을 깨끗이 지우고 새 역사를 씁니다.

`ADR-0002 채택 (TBD-01)`

폴더 탐색기 (클릭하여 상세 보기)

전체 5개 핵심 영역

- mini-react (프로젝트 루트)
  - src
    - core (1단계 핵심 가상 DOM 엔진)
      - vnode.ts ADR-0001
      - dom.ts ADR-0003
      - diff.ts ADR-0004
      - patch.ts ADR-0005
      - history.ts ADR-0002
    - component (2단계 리액티브 훅 계…
      - h.ts FR-12, FR-13
      - function-comp… ADR-0007
      - hooks.ts FR-16~21
      - scheduler.ts ADR-0006
    - pages (데모 & 검증 화면)
      - phase1… FR-02, FR-03, FR-08
      - phase2/App.ts FR-23
  - tests (7계층 검… ADR-0008, ADR-0009
    - unit (VT-01 ~ VT-35)
    - arch (아키텍처 규칙 검 L2 하네스

왼쪽 폴더 트리에서 파일이나 디렉토리를 선택해주세요.

각 파일이 맡고 있는 역할과 직관적인 개념 비유, 아키텍처 철칙을 즉시 확인할 수 있습니다.

발표 시 이 장표에서 꼭 말해야 하는 1문장 팁

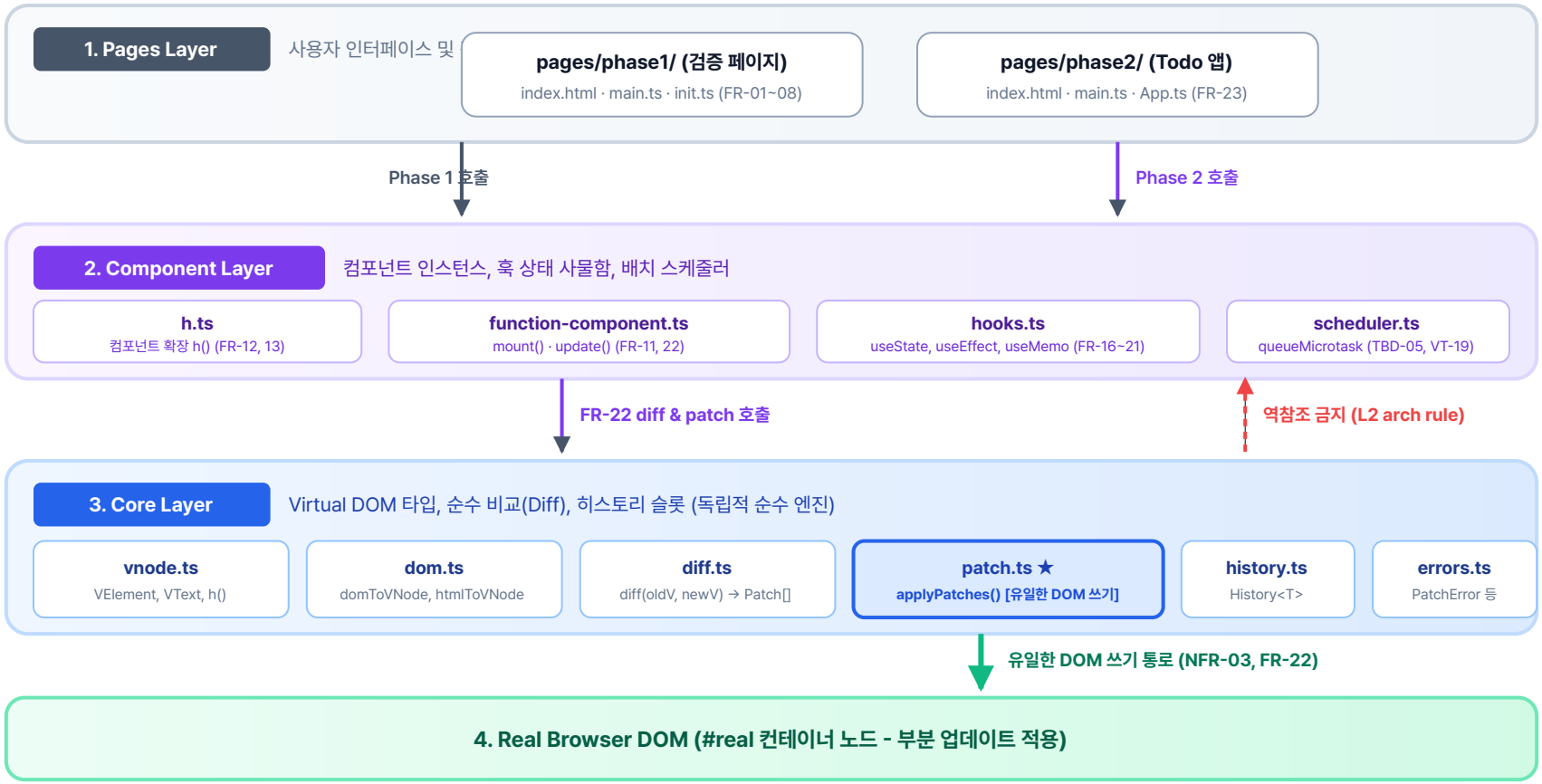
"저희 폴더 구조의 가장 큰 자랑은 **'역할의 명확한 격리'**입니다. 코어 엔진은 DOM을 모르고, 컴포넌트는 재렌더링만 계산하며, **오직 patch.ts 단 1명만 브라우저 화면을 두드립니다.** 이 질서 덕분에 복잡한 프레임워크에서도 버그가 절대 번지지 않습니다."

2장 아키텍처 개요

시스템 계층 구조 및 단방향 의존성 흐름 (Architecture & Dependency)

의존성은 `pages → component → core → DOM` 한 방향으로만 흐르며, core는 component를 모릅니다 (NFR-03, FR-22).

ISO/IEC/IEEE 42010 Architecture Standard



아키텍처 강제 규칙 (depcruise & ESLint)

L2-01 **core-is-independent:** `src/core`는 `src/component`나 `src/pages`를 절대 import할 수 없음.

L2-02 **patch-is-only-writer:** `appendChild`, `remove`, `replaceWith` 등 DOM 변형 API는 오직 **`core/patch.ts`**에서만 호출 가능.

L2-03 **no-innerhtml:** 초기화 1회를 제외하고 `innerHTML` 대입 전면 금지 (ESLint `no-restricted-syntax`).

모듈별 책임과 의존성 요약표

모듈	주요 책임	DOM 접근	의존 대상
`core/vnode`	VNode 타입 정의, h() 생성	없음 (순수 데이터)	없음
`core/dom`	DOM ↔ VNode 양방향 변환	읽기, createElement	core/vnode
`core/diff`	old vs new 비교 (순수함수)	없음	core/vnode
`core/patch`	**Patch[]를 실제 DOM에 적용**	**쓰기 (유일)**	**vnode, dom, errors**
`component/*`	함수 컴포넌트, 훅, 스케줄러	없음 (patch 경유)	src/core

03 3장 Diff 비교 규칙 트리 & Patch 실행 순서도 (Diff & Patch)

3장 Phase 1 파이프라인

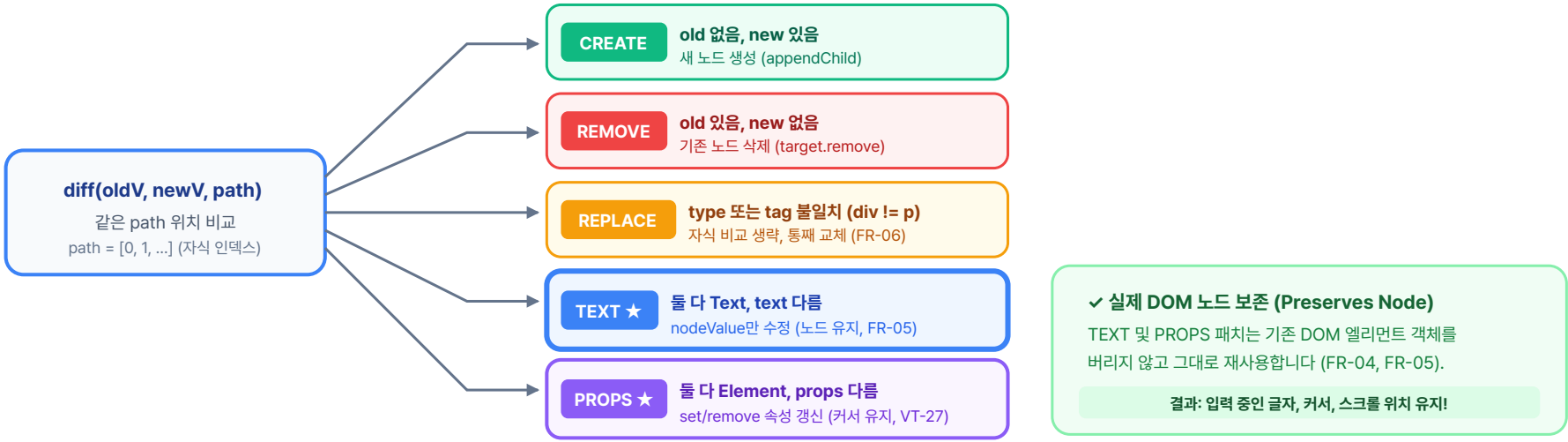
Diff 비교 규칙 트리 및 Patch 버튼 실행 시퀀스 (Diff & Patch Execution Flow)

5대 패치 유형(CREATE, REMOVE, REPLACE, TEXT, PROPS)과 2단계 원자적 적용(Resolve → Apply)

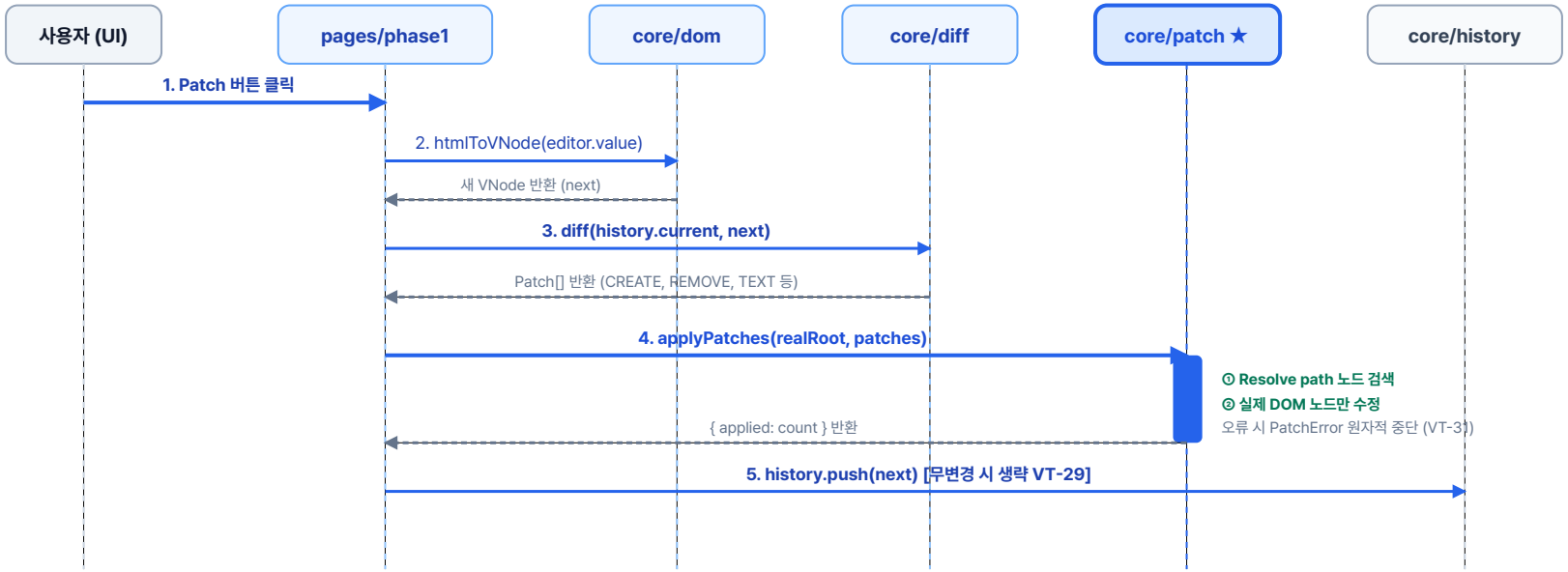
노드 객체 보존 (FR-04, FR-05)

1. Diff 알고리즘 비교 규칙 트리 (동일 path 자식 인덱스 기반 깊이 우선)

클릭하여 세부 명세 확인



2. Patch 버튼 클릭 실행 순서도 (Sequence Flow)



선택된 패치 상세 명세

TEXT

발생 조건:
둘 다 Text 노드이며 내용이 다름

실제 DOM 조작:
`target.nodeValue = text`

기존 노드 객체 보존:
✓ 노드 재사용 (커서/스크롤 유지)

관련 요구사항 & VT:
FR-05, VT-03

04 3.5장 History 세이브 슬롯 및 분기 처리 정책 (Save Slots & Branch Policy)

3.5장 History 세이브 슬롯

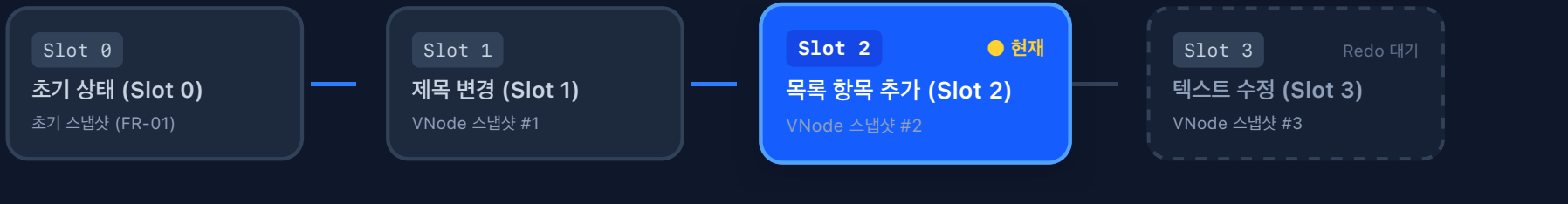
History<VNode> 상태 전이 및 분기 정책 (Save Slot Timeline & Branch Policy)

단일 선형 배열 유지, 분기 시 이후 슬롯 자동 절단(Prune), 변경 없는 패치 미기록 (ADR-0002, VT-29, VT-30)

최대 한도 100개 (FIFO 퇴출)

인터랙티브 시뮬레이터: 버튼을 클릭하여 세이브 슬롯 포인터 이동과 분기 삭제를 체험하세요.

Undo (뒤로) | Redo (앞으로) | 새 Patch 적용 (분기 생성) | 리셋



현재 Slot 2(목록 항목 추가)를 보고 있습니다. Undo나 New Patch를 클릭해 보세요.

상황 1: 정상적인 시간 여행 (Undo & Redo)

기록 배열은 그대로 두고 커서(currentIndex)만 좌우로 이동합니다.



상황 2: Undo 후 새로운 Patch 발생 시 (분기 처리 정책 ADR-0002)

Slot 1에서 새 패치 시 기존 Slot 2는 영구 삭제되며, 새로운 Slot 2(C')로 교체되어 단일



왜 분기 슬롯을 삭제할까? (ADR-0002)

1. 브라우저의 '뒤로 가기 후 새 URL 접속'과 동일한 직관성
2. 복잡한 트리 구조 대신 단일 선형 배열로 Redo 의미가 명확
3. VT-30 경계 조건(처음/끝 비활성화) 검증 간결화

초기 상태 정책 (FR-07)

앱 시작 시의 DOM 스냅샷이 0번 기록이 됩니다. 패치를 n번 하면 기록 수는 항상 **n + 1**개가 됩니다.

무변경 패치 미기록 (VT-29)

diff 결과가 빈 배열 `[]`인 패치는 History에 추가되지 않으며 화면 변화도 발생하지 않습니다.

버튼 비활성화 규칙 (#26)

`index === 0`이면 Undo 비활성, `index === size - 1`이면 Redo가 자동으로 비활성화됩니다.

05 4장 Phase 2 리액티브 루프 & 훅 사물함 (Reactive Loop & Hooks)

4장 Phase 2 리액티브 훅 엔진

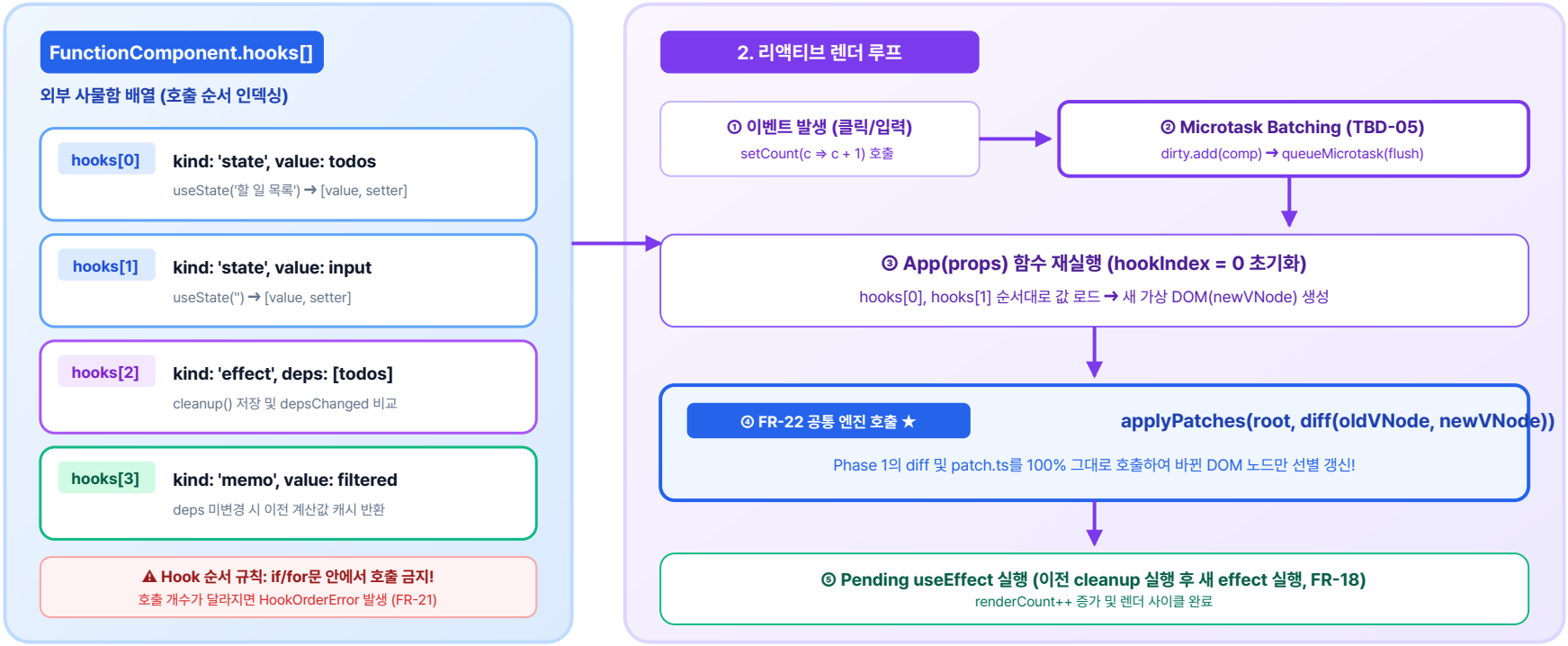
리액티브 렌더 루프 및 훅 사물함 슬롯 (Reactive Loop & Hook Locker)

FR-22 Phase 1 Diff/Patch 공통 재사용

"함수는 매번 새로 실행되는데 상태는 어떻게 유지될까?" — 사물함 배열과 마이크로태스크 배치

LMS 핵심 질문: "함수는 매번 새로 실행되는데 상태는 어떻게 유지될까?"

부품 함수 안의 지역 변수는 함수가 끝나면 사라집니다. 미니 React는 상태를 함수 안이 아니라 부품마다 할당된 외부 **사물함 줄(hooks[] 배열)**에 저장합니다. 함수가 다시 실행될 때 `useState`를 호출한 **순서(0번, 1번...)**대로 사물함을 열어 지난 값을 그대로 꺼내옵니다.



훅 상세 스펙: **useState** useEffect useMemo

최초 렌더:
`initial`(함수면 실행 결과)을 사물함에 저장하고 `[value, setter]` 반환.

재렌더 & 세터:
`Object.is`로 이전 값과 같으면 무시, 다르면 상태 갱신 후 `scheduleUpdate` 예약.

검증 요구사항:
FR-16, FR-17, VT-12, VT-13

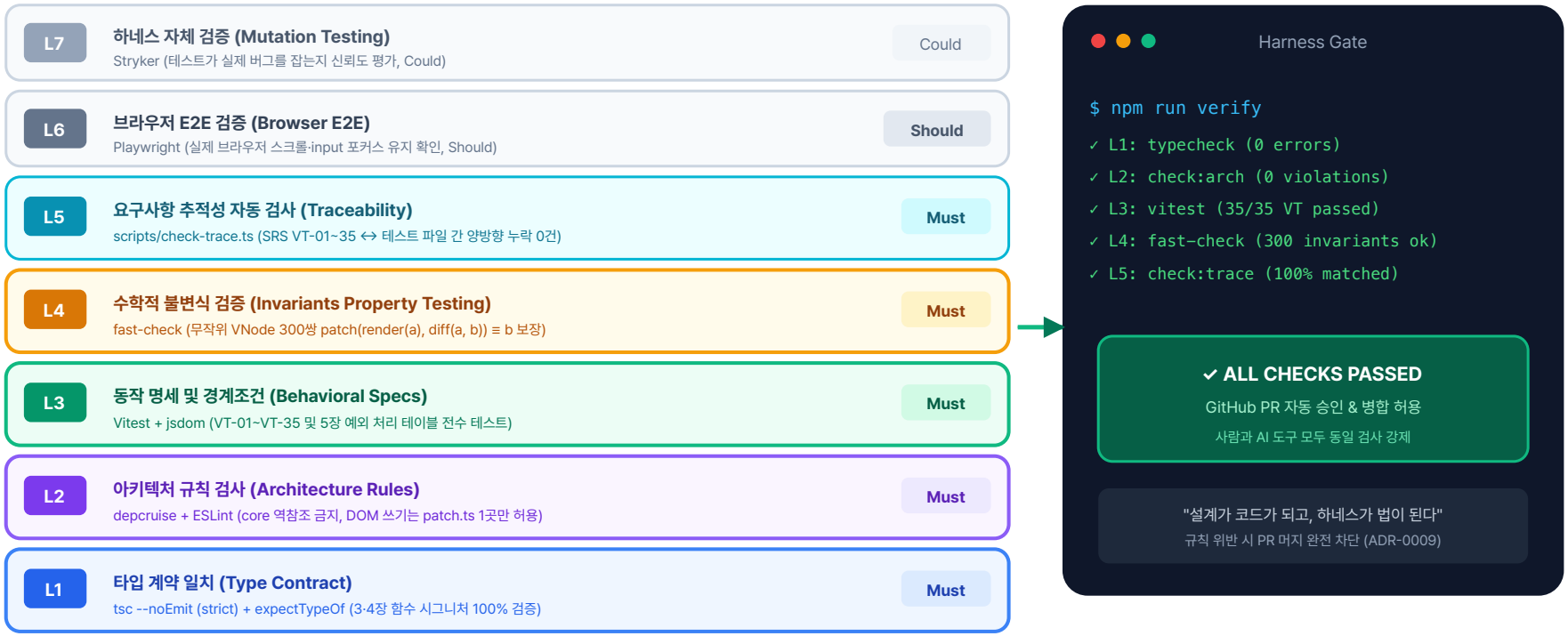
06 8장 7계층 설계 검증 하네스 구조도 (Verification Harness)

8장 설계 검증 하네스

7계층 설계 검증 하네스 구조도 (7-Layer Verification Harness Architecture)

단일 명령어 `npm run verify`로 타입, 아키텍처, 기능, 불변식, 추적성을 일괄 검증

모든 PR 필수 통과 조건 (ADR-0009)



계층	검증 규칙	도구	실행 명령	우선순위
L1	타입 계약 (Type Contract)	tsc --noEmit (strict), Vitest expectTypeOf	npm run typecheck	Must
L2	아키텍처 규칙 (Architecture Rules)	dependency-cruiser, ESLint no-restricted-syntax	npm run check:arch	Must
L3	동작 명세 (Behavioral Spec)	Vitest + jsdom	npm run test	Must
L4	수학적 불변식 (Invariants Property Testing)	fast-check (랜덤 VNode 수백 쌍)	npm run test (prop)	Must
L5	요구사항 추적성 (Traceability Check)	scripts/check-trace.ts	npm run check:trace	Must
L6	브라우저 E2E 검증	Playwright	npm run test:e2e	Should
L7	하네스 자체 검증 (Mutation Testing)	Stryker	npm run test:mutation	Could

07 7장 요구사항 추적성 매트릭스 (FR ↔ SDD ↔ VT)

7장 요구사항 추적성

FR ↔ SDD 설계 모듈 ↔ VT 검증 추적성 매트릭스 (Requirements Traceability Matrix)

ISO/IEC/IEEE 29148 요구사항 공학 기준: 요구사항부터 코드, 테스트 케이스, ADR 결정까지 누락 없는 추적성 보장

FR 21/21 (100%) 매핑 완료

1. SRS 요구사항 (FR) → 2. SDD 설계 모듈 (src/*) → 3. 자동 검증 (VT-01~35) → 4. 아키텍처 결정 (ADR)

전체 보기 (22) | Phase 1 전용 (8건) | Phase 2 전용 (14건)

FR 번호, 모듈명, VT 검색...

요구사항 ID	기능 명칭	설계 모듈	SDD 절	검증 테스트 (VT)	관련 ADR
FR-01	**실제 DOM 읽기 & 초기화** 실제 DOM을 domToVNode로 변환하고 초기 VNode 스냅샷 생성	core/dom.ts	3.2	VT-01, VT-33	ADR-0003
FR-02	**테스트 영역 입력 & 미리보기** HTML 코드 입력칸(textarea)에서 htmlToVNode로 파싱하여 미리보기 제공	core/dom.ts	3.2, 3.6	VT-02, VT-34	ADR-0003
FR-03	**Patch 버튼 실행 파이프라인** diff(history.current, next) -> applyPatches -> history.push 실행	pages/phase1, core/diff.ts	3.6	VT-03, VT-06	ADR-0004

FR-04	**부분 업데이트 & 포커스/스크롤 유지** 바뀐 노드만 수정하고 나머지는 동일 DOM 객체 유지 (input 커서/스크롤 유지)	core/diff.ts, core/patch.ts	3.3, 3.4	VT-04, VT-06, VT-20	ADR-0004
FR-05	**TEXT 및 PROPS 패치** nodeValue 변경 및 setProp/removeAttribute 적용	core/diff.ts, core/patch.ts	3.3, 3.4	VT-05, VT-27	ADR-0005
FR-06	**REPLACE 패치** 태그나 노드 타입이 다르면 하위 비교 없이 새 엘리먼트로 교체	core/diff.ts, core/patch.ts	3.3, 3.4	VT-04	—
FR-07	**History 세이브 슬롯 기록** 스냅샷 저장 (무변경 패치는 기록 안 함), 최대 100개 제한	core/history.ts	3.5	VT-07, VT-29	ADR-0002
FR-08	**Undo / Redo 시간 여행** 특정 VNode로 이동 후 diff/patch로 실제 영역 및 textarea 동기화	core/history.ts, pages/phase1	3.5, 3.6	VT-08, VT-09, VT-30	ADR-0002
FR-11	**함수 컴포넌트 마운트** 최초 렌더링 시 createElement로 DOM 생성 및 container에 마운트	component/function-component.ts	4.2	VT-10	—
FR-12	**컴포넌트 VNode 확장 h()** tag가 함수면 즉시 실행하여 VNode로 펼침	component/h.ts	4.1	VT-11	—
FR-13	**부모-자식 Props 전달** props와 children을 자식 함수 컴포넌트의 인자로 주입	component/h.ts	4.1	VT-11	—
FR-14	**상태 변경 및 재렌더 스케줄링** setter 호출 시 scheduleUpdate로 microtask에 재렌더 등록	component/scheduler.ts	4.4	VT-13	ADR-0006
FR-15	**루트 컴포넌트 단일 상태 소유** 상태는 루트만 소유하고 자식은 Props만 수신하는 순수 함수	component/function-component.ts	4.0	VT-11	ADR-0007
FR-16	**useState 훅 구현** 사물함(hooks 배열)에 값 저장, [value, setter] 반환	component/hooks.ts	4.3	VT-12	—

FR-17	**상태 세터 및 불변성 검사** Object.is로 같으면 무시, 다르면 상태 갱신 후 재렌더 예약	component/hooks.ts	4.3	VT-13	—
FR-18	**useEffect 라이프사이클** 렌더 직후 이전 cleanup 실행 -> 새 effect 실행	component/hooks.ts	4.3	VT-14	—
FR-19	**depsChanged 의존성 비교** Object.is 얕은 비교로 변경 여부 판단	component/hooks.ts	4.3	VT-14	—
FR-20	**useMemo 연산 캐싱** deps 변경 시에만 factory 재실행, 미변경 시 캐시 반환	component/hooks.ts	4.3	VT-15	—
FR-21	**Hook 호출 규칙 런타임 검사** 컴포넌트 밖/자식/조건문 내 호출 감지 시 HookCallError/HookOrderError	component/hooks.ts	4.6	VT-16	—
FR-22	**Phase 1 Diff/Patch 엔진 공통 재사용** update() 내부에서 Phase 1 core diff/applyPatches를 그대로 사용	component/function-component.ts	4.2	VT-17	—
FR-23	**Todo App 통합 검증 페이지** 할 일 목록 앱으로 State, Props, Hooks, Event, Batching 종합 검증	pages/phase2/App.ts	4.7	VT-18	—
TBD-05	**Microtask 기반 Batching** 한 이벤트에서 setter 3회 호출해도 renderCount는 1회만 증가	component/scheduler.ts	4.4	VT-19	ADR-0006

비전공자 & 입문자를 위한 소프트웨어 공학 용어 해설 가이드

컴퓨터 공학을 전공하지 않아도 한눈에 이해할 수 있도록 일상적인 비유로 쉽게 풀었습니다.

FR (기능 요구사항) ≒ "고객의 주문서"

Functional Requirement의 약자입니다. "프로

SDD (설계 명세서) ≒ "건축 설계도"

Software Design Description의 약자입니다.

VT (검증 테스트) ≒ "합격/불합격 시험지"

Verification Test의 약자입니다. 개발자가 만든

그램이 마땅히 해야 하는 일" 목록입니다. (예: "버튼을 누르면 화면이 바뀌어야 한다", "실행 취소(Undo)를 누르면 1칸 전으로 되돌아가야 한다")

주문서(FR)를 받고 나서, 실제로 어떤 폴더의 어떤 파일(`src/core/diff.ts` 등)이 그 일을 처리할지 그림과 표로 꼼꼼히 설계한 도면입니다.

프로그램이 주문서(FR)대로 거짓 없이 잘 돌아가는지 컴퓨터가 0.1초 만에 자동으로 채점하는 35가지 시험 문항입니다 (VT-01 ~ VT-35).

추적성 매트릭스 ≒ "주문서 ↔ 영수증 대조표"

"주문서에 있는 기능(FR)을 하나도 빼먹지 않고 설계도(SDD)에 넣었는가? 그리고 그 기능을 진짜 시험(VT)으로 확인했는가?"를 한 줄씩 1:1로 대조하여 **누락율 0%를 증명하는 체크 표**입니다.

ISO 29148 기준 ≒ "국제 표준 품질 마크"

전 세계 공학 전문가들이 모여 "소프트웨어를 만들 때 요구사항을 어떻게 관리해야 버그 없이 안전한가?"를 정해둔 공식 국제 표준 규격입니다. 저희는 이 표준에 맞춰 100% 매핑을 달성했습니다.

ADR & 하네스 ≒ "결정 일기장 & 안전망"

ADR은 "A로 할까 B로 할까?" 고민했을 때 선택한 9가지 기술 일기장이며, **하네스**는 규칙을 어긴 코드가 들어오면 자동으로 막아주는 7겹의 번지점프 안전로프(`npm run verify`)입니다.

부록 A 부록 A. 파일별 함수 명세표 (Issue #10) — 30개 함수 전수

파일	함수명	받는 것 (입력)	돌려주는 것 (출력)	하는 일 (쉬운 설명)	요구사항
1. Virtual DOM 및 DOM 변환 (core/vnode, core/dom)					
core/vnode.ts	h(tag, props, ...children)	태그, 속성, 자식	VElement	설계도 조각 하나를 만든다	FR-01
core/dom.ts	domToVNode(node)	화면의 노드	VNode | null	진짜 화면을 보고 설계도를 그린다	FR-01
core/dom.ts	htmlToVNode(html, rootTag)	HTML 글	VElement	입력칸의 글을 설계도로 바꾼다	FR-02
core/dom.ts	createElement(vnode)	VNode	Node	설계도대로 새 블록을 만든다	FR-06
core/dom.ts	vnodeToHTML(vnode)	VNode	string	설계도를 다시 글로 바꾼다 (Undo 후 입력칸 갱신)	FR-08
2. 비교 및 패치 적용 (core/diff, core/patch)					
core/diff.ts	diff(oldV, newV, path)	옛 설계도, 새 설계도	Patch[]	틀린 그림을 찾아 목록으로 만든다	FR-03~06
core/diff.ts	diffProps(oldP, newP)	옛 속성, 새 속성	{ set, remove }	바뀐 속성과 사라진 속성만 고른다	FR-05
core/patch.ts	**applyPatches(root, patches)**	실제 영역, Patch[]	**{ applied }**	**목록대로 바뀐 곳만 고친다. 오류 시 전체 롤백**	FR-03, 04
core/patch.ts	setProp(el, name, value)	요소, 속성명, 값	void	속성 변경. 값이 같으면 쓰지 않아 커서 보존	FR-05
3. 히스토리 타임머신 (core/history)					
core/history.ts	push(state)	새 상태	void	새 슬롯에 저장. 분기 시 뒤쪽 슬롯 삭제	FR-07
core/history.ts	undo() / redo()	없음	T | null	한 칸 뒤로 / 앞으로 이동. 끝이면 null	FR-08
core/history.ts	canUndo() / canRedo()	없음	boolean	버튼 활성/비활성화 상태 반환	FR-08 (#26)

4. Phase 2 컴포넌트 & Hooks (component/*)					
component/function-component.ts	render(fn, props, container)	루트부품, Props, 자리	FunctionComponent	앱을 처음 화면에 올린다	FR-11, 12
component/hooks.ts	useState(initial)	초기값	[value, setter]	상태 사물함 슬롯 생성 및 세터 반환	FR-16, 17
component/hooks.ts	useEffect(effect, deps)	콜백함수, 의존성 배열	void	렌더 직후 사이드이펙트 예약 실행	FR-18, 19
component/hooks.ts	useMemo(factory, deps)	연산함수, 의존성 배열	T	deps 변경 시만 재계산, 캐시 반환	FR-20
component/scheduler.ts	scheduleUpdate(component)	컴포넌트 인스턴스	void	queueMicrotask로 상태 변경 모아 1회 렌더	FR-14, TBD-05

부록 B 부록 B. 원리 설명 질문 카드 (핵심 질문 15종)

Q1. React는 왜 만들어졌을까? 화면을 통째로 다시 그리면 무엇이 문제일까?
답: DOM 조작 비용 증가로 느려지고, input 글자·커서·스크롤 위치가 초기화되기 때문 (0장).

Q2. 실제 화면을 바꾸는 코드는 왜 한 파일(core/patch.ts)에만 있어야 할까?
답: 단방향 의존성 및 단일 책임 원칙 준수, DOM 변형 추적 및 하네스 L2 검증 용이 (2장).

Q3. 공백만 있는 글자를 왜 버릴까? 안 버리면 무슨 일이 생길까?
답: HTML 들여쓰기 공백이 text 노드로 변환되면 실제 DOM과 VNode의 path 인덱스가 밀림 (3.2).

Q4. 패치를 하나씩 바로 적용하지 않고 먼저 전부 찾아 두는(Resolve) 이유는?
답: 노드 삭제 시 뒷순서 인덱스가 밀리는 현상(Index Drift) 방지 및 사전 원자적 검증 (ADR-0005).

Q5. Hook을 if문 안에서 부르면 왜 값이 섞일까?
답: Hook은 이름이 아니라 호출 순서(0번, 1번...)로 외부 사물함(hooks[])을 찾기 때문 (4.6).

Q6. 버튼 한 번에 값을 3번 바꾸면 화면은 몇 번 다시 그려질까?
답: 1번만 다시 그려짐. queueMicrotask 기반 Batching이 1회로 병합 (ADR-0006, VT-19).

Mini-React 프로젝트 팀 · 소프트웨어 설계 문서 (SDD v0.3)

본 문서는 IEEE 1016-2009 및 ISO/IEC/IEEE 29148 요구사항 공학 표준을 준수하며, 단일 명령 `npm run verify`로 검증됩니다.